

讓數字變成會動的指針

前幾週把資料顯示在螢幕上。這週讓同一個數字，也能用位置表示。

程式中的同一個 `count = 3`



成果示意：數字是格數，不是實測角度。

先在電腦輸入 0，再輸入 1。確認供電與接線後，觀察指針是否移到下一格。

今天做的是**輕量紙指針**，不是推門、吊重物或讓車輪一直轉。位置由你輸入，不是自動偵測人數。

今天的完成條件

- 說得出電源線、訊號線、GND 各接哪裡。
- 讓數字、OLED 與紙指針表達同一個值。
- 能中止操作，分清「停訊號」與「切電源」。

講義含參考解答與完整程式。先一起操作、觀察結果，再回來讀原理；程式附錄不必逐頁講授。

不用從 PDF 一頁一頁抄程式

1. 已有教師提供的完整 IoT 資料夾，就直接使用同版本資料夾，不重新下載另一版。
2. 沒有資料夾時，開啟 [課程 GitHub](#)，按 **Code** → **Download ZIP**。下載完成後在檔案總管右鍵 ZIP，選「全部解壓縮」。
3. 打開解壓縮後的資料夾，再開 **IOT_Introduction**。不要直接在 ZIP 預覽視窗操作。
4. 依下表找到當段程式；在 Arduino IDE 選「檔案 → 開啟」，選該資料夾中的同名 **.ino**。瀏覽器裡看到程式文字，不等於已在 IDE 開啟。
5. 修改前選「檔案 → 另存新檔」，存進自己的練習資料夾。按 **Ctrl+F** 搜尋講義指定的設定名稱，改等號右邊的值，保留分號，再按 **Ctrl+S**。

本週依序開這些檔案

以下從 IOT_Introduction 資料夾開始找；每次只開當段要用的檔案。

- docs/teaching_drafts/week6_redesign/count_preview/count_preview.ino
- examples/week06_servo_pointer/week06_servo_pointer.ino
- docs/teaching_drafts/week6_redesign/serial_pointer_counter/serial_pointer_counter.ino

換另一支程式時，原來修改的設定**不會自動帶過去**。依該段設定表逐項填寫；不把短片段當成完整程式。

若下載內容和講義列出的檔名不一致，先取得教師正在使用的同版本資料夾，不猜替代檔。附錄是查閱用；本頁只準備檔案，尚未上傳或通電。

一次只增加一件事

順序	一起做什麼	看到什麼再往下
1	裸板輸入數字，約 15 分鐘	Serial 顯示對應的示例角度
2	看實物、量電源，約 30 分鐘	找到端子、量測位置與允許範圍
3	接停止鍵與舵機，約 35 分鐘	停止鍵有效，受限位置平穩
4	做紙指針、再接 OLED，約 25 分鐘	畫面與實際位置分開核對
5	加減計數、改造題，約 25 分鐘	邊界、太快輸入與中止有紀錄

另留約 15 分鐘問答、收線與記錄；休息依現場安排。速度較慢時，加減改造作延伸，不省略斷電和停止檢查。

先說清楚一個限制

舵機確切規格、負載供電及啟動行為，必須在課前由授課者完成驗證並提供設定表。**這份講義沒有替實物完成驗證。**

若設定表尚未齊全，今天先做數字預覽、斷電辨識與題目，不把模擬結果說成舵機已完成。這不是要求每組排隊取得教師簽名，而是全班使用同一份已確認、且與自己器材相符的接法。

先讓電腦顯示一個結果

1. 電池盒 OFF，取出至少一顆電池；拔 USB。
2. 暫時移除所有杜邦線與模組，只留下 ESP32 開發板。
3. 依前面的檔案表開啟 `count_preview.ino`，連接 Week 2 已用過的板背 COM 接頭。其他板型以實物接頭為準。
4. 在 Arduino IDE 選擇 Week 2 已成功使用的板型、Port，按勾號編譯，再按右箭頭上傳。
5. 開啟右上角 Serial Monitor（序列監控視窗），選 115200。
6. 輸入 3，按 Enter。再分別輸入 0、6。

此程式只做數字運算，不設定 GPIO，也不輸出舵機控制訊號。尚未完成 Week 2 上傳者，先回到該週 IDE 設定頁，不猜 Port。

預期格式如下，這是**示例輸出**，不是實機量測：

```
mode=preview count=3 angle_example=90  
servo_connected=false
```

實際程式將上面欄位印在同一行。輸入框請一次送一個數字；**10** 是兩個字元，不是「第十格」。

0~6，不是 0~6 度

這個數字預覽假設：第 0 格對應 60，第 6 格對應 120。兩端之間有 6 個間隔。

每格差 = $(120 - 60) / 6 = 10$
示例角度 = $60 + \text{格數} \times 10$

格數 count	0	1	2	3	4	5	6
示例角度	60	70	80	90	100	110	120



這叫做**映射**：把一組數字對應到另一組數字。現在只在記憶體中算出結果，沒有舵機動作。

60~120 只是本頁算式的示例，不是你手上舵機的安全角度。後面的實機程式必須使用課前確認的數值。

先預測輸入 4 會出現什麼，再執行檢查。輸入 9 應被拒絕，不擴大範圍。

桌上只留今天需要的東西

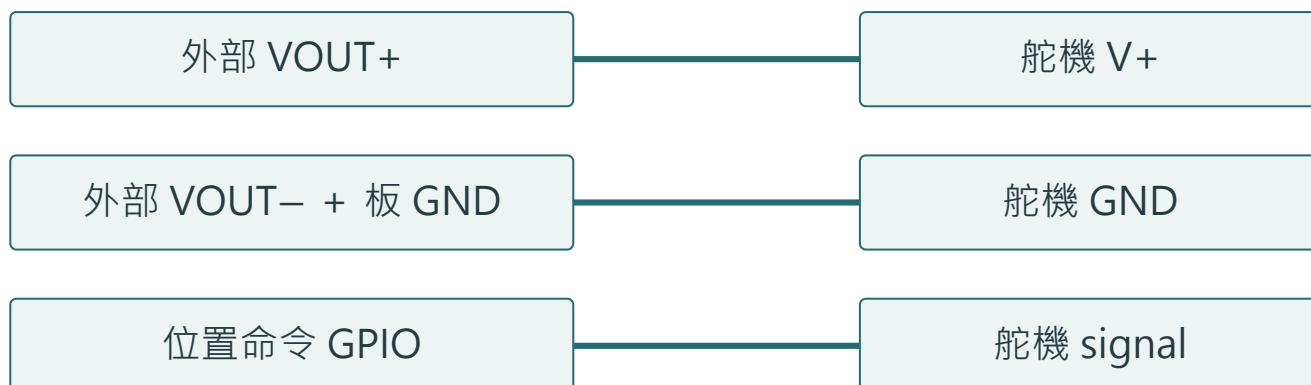


課程既有 SG90 實物照片。外觀與線色不能代替版本及腳位確認。

東西	今天做什麼
ESP32、USB、電腦	執行程式、發出命令、顯示紀錄
定位型 SG90、配套舵盤	改變指針的位置
四槽電池盒、降壓模組、安全端子	供應舵機工作電力
已辨認的按鈕、麵包板、訊號線	按下即要求停止
電表、短紙指針、固定底板	檢查電壓、固定及觀察
Week 5 已確認的 OLED	後半段才加入

先收起 RGB、DHT、KY 與蜂鳴器。這週不需要 L298N。不能把連續旋轉型舵機當作定位型使用。

舵機的三條線各做一件事



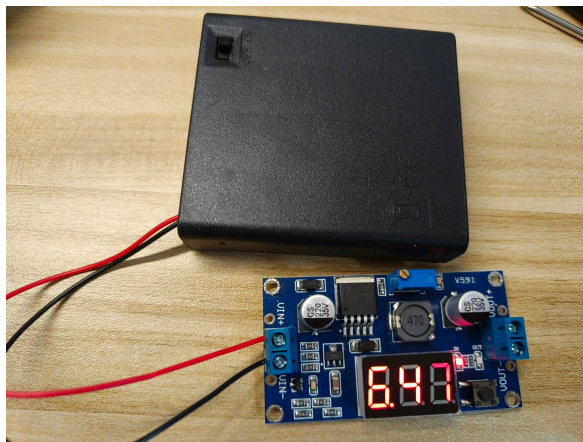
按功能排列，不代表實物三針左右順序。

功能	白話意思	接到哪裡
V+	提供工作電力	外部降壓輸出正極
GND	電力回路與訊號參考	外部輸出負極，並與 ESP32 GND 共接
signal	接收位置指令	已確認可用的 GPIO

GPIO 是「告訴它要去哪裡」，不是提供轉動所需的全部電力。**不要把 V+ 接到 GPIO 或 3V3。**

先把實物的三個功能指出來，才開始接線。照片上的顏色只是辨識線的輔助；功能以確切版本文件與已核對的接頭方向為準。

先不要接舵機



教師既有照片：VIN 接電池，VOUT 尚未接舵機。IN 顯示不是輸出電壓。

全班先在斷電狀態一起找四個端子：VIN+、VIN-、VOUT+、VOUT-。

1. 電池盒 OFF，取出至少一顆電池。
2. 電池正線固定到 VIN+，負線固定到 VIN-。
3. VOUT 暫時不接舵機；正、負導體各自絕緣，不互碰。
4. 輕拉確認固定，沒有裸銅鬚跨到旁邊端子。

端子若不匹配，先停在辨認步驟，不把裸線塞入杜邦母頭，也不靠膠帶維持鬆脫接點。

降壓模組把較高電壓降下來，不是升壓器，也不是電池。顯示數字正常，不代表已能帶動舵機。

黑筆接哪裡？紅筆接哪裡？



電表並聯跨兩端；不是串成供電線。

1. 黑筆插 COM；紅筆插 $V\Omega mA$ ，不插 10A。
2. A830L 轉到直流電壓 20 V 檔，不是電阻或通斷檔。
3. 開關 OFF 時固定黑筆在 VOUT-、紅筆在 VOUT+。避免兩支金屬筆尖互碰。
4. 裝回同種類、同狀態的電池；接點固定後 ON，讀取數字、正負號與 V 單位。
5. OFF，再移開表筆或改線。

本頁先量測，不自行調整。只有課前已提供這組電源的目標輸出、允許上下限與操作方法，才由相符配置進行調整；未取得就 OFF、停止在空載讀值紀錄。4.76 V 是歷史量測，不是本次目標設定。

這是在量兩點的電壓差，不是在量電流。不可把電表切到電流檔，再跨接正負極。

空載讀值不能保證動作時一樣

課程既有紀錄如下，三個值來自不同測點或儀表：

既有回報	它表示什麼	不能直接推論什麼
四顆 1.5 V AA，量到 6.6 V	電池組當時的電壓	舵機可直接接 6.6 V
模組 OUT 顯示 4.85 V	模組自己的顯示值	電表也一定顯示 4.85 V
電表跨 VOUT 量到 4.76 V	當時尚未接舵機的讀值	動作時供電已通過

空載：沒有供應舵機工作電流。**帶載**：舵機已接上並在工作。

兩者都用 V 表示電壓。帶載可能掉壓；慢速電表也可能漏掉短暫變化。4.76 V 不因「很接近 4.8」就自動合格，須對照確切舵機規格、量測不確定度與停止條件。

現行課程方案是四顆 1.5 V AA 加降壓模組；不能直接換成四顆 1.2 V eneloop，再假設仍能穩定降到 5 V。替換電池是另一個需要確認的供電方案。

接線之前，要有這一張表

要填的資料	來源或確認方式
板卡版型、signal GPIO、STOP GPIO	板上絲印及該板可用腳位；兩腳不重複
舵機三線功能、供電範圍、訊號相容性	確切版本文件，不只商品名稱
電源允許範圍、負載能力、停止條件	指定電池、模組、端子與導線的確認結果
頻率、脈寬範圍、起始及七格角度	無外接負載的課前受限測試
上電順序與停止方式	包括工作電源 OFF 時能否接受 signal
OLED 腳位、位址及控制器設定	沿用 Week 5 的實際通過組合

程式中的 false 與 -1 是什麼？

false 表示「尚未確認可以啟用」；-1 表示「尚未填入設定」。它們刻意讓程式先不操作舵機。

若顯示 `status=blocked reason=profile_missing`，意思是設定還缺，不是舵機壞掉。**不要只把所有 false 改成 true 來消除訊息。**

本課先前提出 GPIO7 作 signal 的候選接法，但沒有實機通過紀錄。本頁不把候選值當作全班已驗證配置。

先上傳，再接完整電路

依檔案表開啟 `week06_servo_pointer.ino`，不再使用 `count_preview.ino`。先讀下頁的逐欄設定表，確認資料齊全，再執行本頁。

1. 所有外接線移除、電池 OFF。只接開發板 USB。
2. Arduino IDE 左側「程式庫管理員」搜尋並安裝 ESP32Servo；本教材沿用 3.2.1。
3. 安裝 u8g2，本教材沿用 2.36.15；即使暫時不接 OLED，這支程式仍需要它才能編譯。
4. 依課前設定表填寫程式頂端設定，先保留 `WITH_OLED=false`。
5. 確認使用原先板型、Port 與 Arduino-ESP32 3.3.11。編譯、上傳，開啟 115200 的 Serial Monitor。
6. 拔 USB。接著依「沿用 Week 2 的按鈕」頁接 STOP，再依「舵機接線分兩部分」頁接舵機，不先接完又拆一次。

設定還不完整時，只驗證編譯及 blocked 訊息；不接舵機電源。上傳成功只證明程式已寫入，不證明接線或機構安全。

這次主要只學四種操作

a 準備、0~6 指定格數、q 停訊號、按 STOP 中止。其他復原命令留到第一次停止之後再學。

要填的名稱，和還缺的資料

按 Ctrl+F 搜尋左欄，修改原來那一行，不新增第二份同名變數。兩支舵機程式的下列名稱相同；換到 `serial_pointer_counter.ino` 時逐欄重填。

程式名稱	值的來源與目前限制
DEVICE_ID	例如 "group01"，保留雙引號
PIN_STOP	對應停止鍵；T01 GPIO5 已有按鈕觀察，可作本次配置依據，與其他腳不重複
PIN_SERVO	舵機配置尚未確認；GPIO7 只是候選，不把候選當通過
SERVO_HZ	確切舵機所需頻率，尚待確認
SERVO_MIN_US、SERVO_MAX_US	已驗證脈寬範圍，尚待確認
SAFE_MIN_ANGLE、SAFE_MAX_ANGLE	課前實物允許的命令範圍，尚待確認
ZERO_ANGLE、SIX_ANGLE	第 0 / 6 格命令；在上述範圍內，尚待確認
SERVO_PROFILE_CONFIRMED、 POWER_AND_FIXTURE_CONFIRMED、 STOP_PROFILE_CONFIRMED	三項條件各自完成才 true；現在不整批改 true
WITH_OLED	初次 false，後段接回螢幕才 true

仍缺舵機資料時停在數字預覽與斷電辨識。公開程式保留 -1 / false 是預期保護，不是要求學生自行試出一組能動的數值。補齊接頭照片、供電及小範圍測試記錄後，教師才能把這張表換成該批器材的已填版本。

沿用 Week 2 的按鈕

所有電源斷開，舵機先不接。選按鈕的兩組不同接點；用斷電通斷檔確認「放開不通、按下通」。



這條線	從哪裡	到哪裡
地線	ESP32 GND	麵包板 a3
STOP 訊號	設定表上的 STOP GPIO	a27
回地	a29	c3

按鈕四腳插 **e27**、**f27**、**e29**、**f29**，沿用 Week 2 跨中央溝槽的方向。斷電量 b27 對 b29：放開不通、按下才通。a27 與 e27 相通，a29 與 e29 相通；不是把 GPIO 接到按鈕同一組的兩腳。

先核對麵包板的孔組；板子放旁邊用線連接，避免板體擋住插孔。

為什麼按下會讀到 LOW ？



程式設定 `INPUT_PULLUP`：啟用晶片內部的上拉電阻。

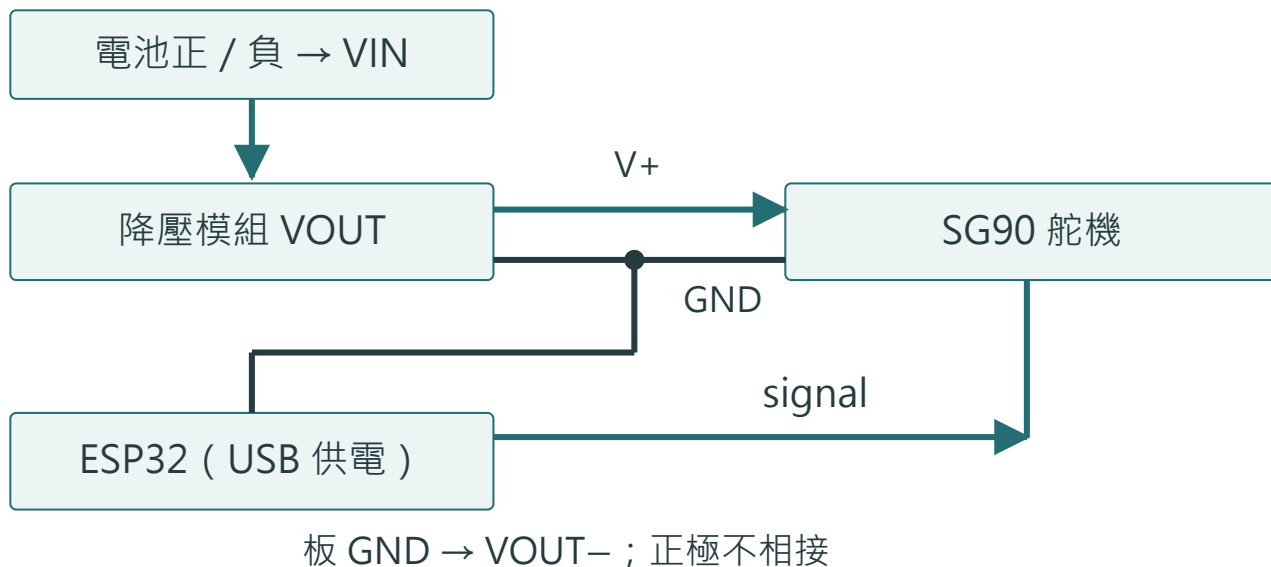
- 放開按鈕：通往 GND 的路斷開，輸入通常讀 HIGH。
- 按下按鈕：GPIO 經按鈕連到 GND，讀 LOW。
- 上拉電阻限制這條回路的電流，因此不是把 3V3 電源直接用線短接到 GND。

這裡的 STOP GPIO 是**輸入**，不能沿用 Week 3 的輸出高低電壓程式。先移除舊線，換上本週程式，再按本週圖接線。

STOP 是軟體讀取的停止鍵，不是工業安全急停，也不是電池開關。程式失效時不能保證它仍有效。

舵機接線分兩部分

USB 拔除，電池 OFF 並取出至少一顆。確認接頭方向，不按線色猜功能。



線路	來源 → 目的
工作電力正極	VOUT+ → 安全端子 → 舵機 V+
工作電力負極	舵機 GND → 安全端子 → VOUT-
共同參考	麵包板 d3 → 外部負極端子；a3 已接板 GND，不再塞第二條線
位置指令	設定表的 signal GPIO → 舵機 signal

舵機工作電流不經麵包板的低功率電源列。端子與導線須符合負載需求，接頭固定不能鬆脫。

「安全端子」不是指把裸線塞入杜邦孔：課前需備好與舵機母接頭匹配、已核對極性的公頭轉接線，以及適合該線徑的絕緣接線端子。正端接 VOUT+ 與舵機 V+；負端接 VOUT-、舵機 GND 與 d3。目前尚無已確認轉接件照片，**沒有相符配件時不組裝此段**。

外部正極不接 ESP32 的 GPIO、3V3、5Vin 或 USB VBUS。只將指定的 GND 接在一起，不是把兩個電源正極相接。

先看停止鍵的紀錄

本頁須已有完整、相符的設定。舵機外部電源仍 OFF；接上板子的 USB。

操作	預期狀態或紀錄
開機，不按鍵	DISARMED，尚未啟用控制脈波
按住 STOP	ABORTED、physical_stop
按住時送 a	不能啟用
放開，稍等後送 c	清除可清除的原因，仍不動
再送 z	回到 DISARMED，仍不動

看到 DISARMED，白話是「尚未準備控制」；ABORTED 是「已中止，需要處理原因」。

這段只檢查按鈕與程式狀態，不代表舵機轉動正常。尚未確認工作電源 OFF 時接收訊號是否相容，不能送 a。

STOP 若沒反應，先斷電檢查腳位、按鈕接點及目前上傳的程式，不進入下一步。

先不裝紙指針

本頁僅適用已完成課前供電、接頭、脈寬及上電順序驗證的器材。固定舵機本體，轉軸周圍沒有手、導線或其他物件。

1. 外部電源 OFF，USB 已接，狀態 DISARMED，STOP 放開。
2. 在已確認允許先有 signal、再有工作電源的配置，送 a。程式準備第 0 格的命令。
3. 確認紀錄後，把外部電源 ON；觀察起始位置。
4. 平穩後才送 1，和上一個命令至少隔 1 秒。初次只走已確認的小範圍。
5. 正常結束時送 q，外部 OFF，再拔 USB。

a 的紀錄只是操作者表示「我已關外部電源」，**程式沒有量到開關是否真的 OFF。**

這個先後順序不是所有舵機都適用。若可能由 signal 回灌、啟動亂動或規格未明，保持斷電；不要自行改上電順序試運氣。課前須先解決，不能只靠程式保護旗標。

有卡住、抖動、發熱、異味或板子重啟，立即外部 OFF，再拔 USB。不等待 30 秒、不提高電壓、不強扳轉軸。

哪些是程式說的？哪些是你看到的？

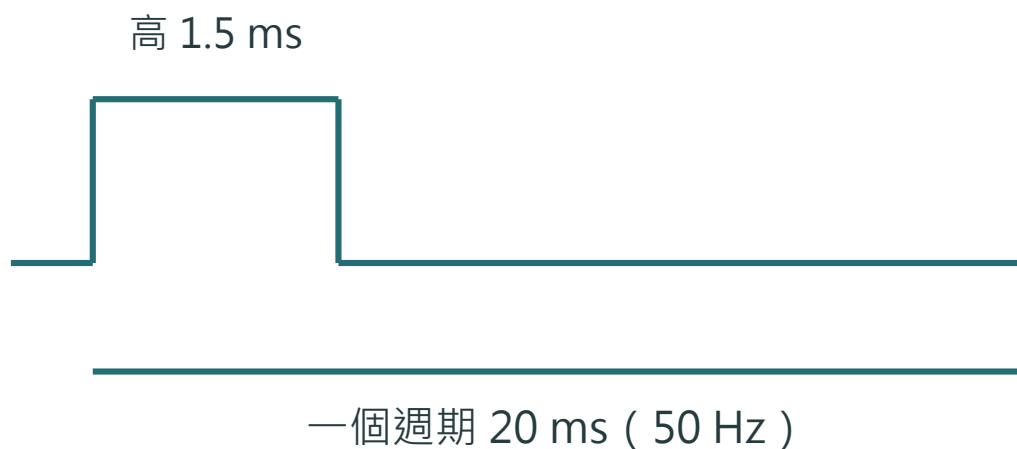
記錄項目	怎麼取得	可以說什麼
count=1	Serial	程式接受第 1 格
angle_command	Serial	程式送出的角度命令
軸是否移動、是否平穩	觀察 / 影片	該次實際可見反應
舵機 V+ 對 GND 電壓	電表並聯量測	當時量到的電壓

帶載量測先 OFF、拔 USB，固定安全測點，再按已確認流程重新操作。不要在移動機構旁臨時用筆尖找裸線。

電表不能證明沒有短暫掉壓；程式也沒有角度或電流感測。position_measured=false 就是「沒有量到實際位置」。

如果 Serial 有紀錄但軸沒動，先停止，再分開查電源、signal、共同地與機構。不用反覆送指令來掩蓋問題。

GPIO 不是一直送出「90」



時間示意，寬度不按比例；不是實物核准值。

圖中高電位維持 1.5 ms，整段每 20 ms 重複一次。這叫**脈波**；高電位持續的時間叫**脈寬**。

程式動作	它做的事	沒有做的事
<code>attach()</code>	啟動控制通道	不保證機構沒有碰撞
<code>write(angle)</code>	按設定換算脈波	不量測真實角度
<code>detach()</code>	停止控制脈波	不切斷外部電源

20 ms 相當於每秒 50 次；1.5 ms 等於 1500 μ s。這是時間示意，不是本班所有舵機共同安全設定。

一般三線舵機會在內部控制位置，但本程式沒有讀回實際角度；library 的 `read()` 也不能當作外接角度感測器。

兩邊需要同一個電壓參考



Week 3 用電表看「紅筆電壓減黑筆電壓」。舵機判讀 signal 時，也要相對自己的 GND 看高低。

ESP32 說「相對我的 GND 是 HIGH」，不等於隔離的另一個電路自然看到相同電壓。因此按接線圖，把指定 GND 連起來。

兩條路徑分開追

工作電流：外部 VOUT+ → 舵機內部電路與馬達 → 舵機 GND → VOUT- → 電源內部回路。

資訊：Serial 命令 → ESP32 計算 → signal 的脈波 → 舵機內部控制 → 軸移動。signal 相對共同 GND 判讀。

共地不代表把舵機工作電流繞進板上的細訊號線。也不要帶電拔 GND 來展示錯誤。

先對位置，再固定紙片



課程既有配件照片。白色舵盤裝在軸上，紙片固定在舵盤上，不把軸黏死。

1. 先看完五步再操作。接 USB、外部 OFF，若是 ABORTED，先排除原因並送 c、z；看到 DISARMED 且 STOP 放開，依已確認配置送 a，再外部 ON。a 已命令第 0 格，平穩後記下方向。
2. 外部 OFF、拔 USB，才安裝配套舵盤與短、輕、無尖角紙指針。
3. 本體固定在底板；不靠手壓住、不把膠灌進軸或齒輪。
4. 重新接 USB、外部 OFF、STOP 放開，看到 DISARMED 才送 a，再外部 ON。一次只觀察一兩格，命令至少隔 1 秒；送 q、外部 OFF、拔 USB 後，才在紙盤標位置。
5. 每格穩定後才下一格，最後試 4 → 3 的回退。

第 0 格不必朝正上方，也不代表實際 0 度。不要為了對印好的刻度，強行扳軸或增加行程。

七格全範圍未通過前，只標已確認的小範圍；不因講義有數字 6 就直接跳到端點。

每次 a 後最多 30 秒；不要啟用後停下來聽講或拍照。若逾時中止，先外部 OFF，依停止頁 c、z 復原後重新操作，不刪掉逾時保護。

把 Week 5 的顯示設定填回來

先送 q、外部 OFF、取出至少一顆電池、拔 USB。拍照保留剛才的接法，移除開發板外接線，再修改同一支 `week06_servo_pointer.ino`。

搜尋欄名	本次填寫
WITH_OLED	true，表示現在加入螢幕
PIN_SDA	本組 Week 5 已確認值；T01 為 8
PIN_SCL	本組 Week 5 已確認值；T01 為 9
OLED_ADDRESS_7BIT	本組位址；T01 為 0x3C
OLED_CONTROLLER	搜尋 <code>#define OLED_CONTROLLER</code> ；T01 為 1315 相容配置
OLED_PROFILE_CONFIRMED	本組供電及訊號相容性確認後才 true

舵機與 STOP 設定保留，四個訊號 GPIO 不重複。不同螢幕不能只照抄 T01；位址有回應也不等於控制器型號已辨識。

只接 COM 的 USB，Verify、Upload；完成後關 Monitor、拔 USB。下一頁才恢復舊線並新增螢幕供電線，現在不啟用舵機。

同一個數字，兩種呈現

先外部 OFF、取出至少一顆電池、拔 USB。沿用 Week 5 已確認的 OLED，不重新猜控制器。

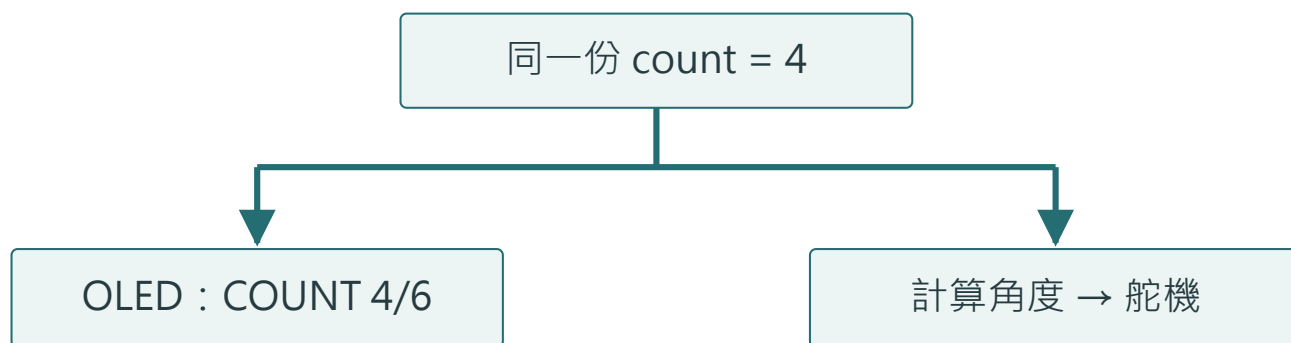
本次新增或核對的線	接到哪裡
ESP32 3V3	a6，這條供電線本週尚未接，現在新增
ESP32 GND	a3，沿用 STOP 接線；不要拔掉共地
GND	b3，與 a3 的 ESP32 GND 同組
VDD / VCC	b6，與 a6 的 ESP32 3V3 同組
SDA	Week 5 已確認的 SDA GPIO
SCK / SCL	Week 5 已確認的 SCL GPIO

SDA、SCL、signal、STOP 四個 GPIO 不得重複。T01 曾用 SDA8 / SCL9、0x3C 與 SSD1315 相容配置有畫面；不是所有同名螢幕都能直接套用。

1. 前頁已更新程式；USB 拔除、外部 OFF，先恢復 STOP 與舵機接線，再依上表新增 3V3 → a6 與 OLED 四線。
2. 每孔只插一個接頭，重新核對外部正極不接板上電源；a3、b3、c3、d3 分別是板地、OLED、STOP、外部共地。
3. 接 USB、外部 OFF、STOP 放開；看到 DISARMED 才依已確認配置送 a、外部 ON，送 1，確認畫面與指針。結束送 q、外部 OFF、拔 USB。

OLED 顯示 NO SENSE 是提醒「沒有量到真實位置」，不是一定壞了。ACK 回應也不證明每個像素都正確。

共用 count，分開檢查結果



螢幕是程式資料；紙指針位置另行觀察。

先說出你預期的結果，再送出一個命令：

命令	OLED 預期	指針應觀察什麼
1	COUNT 1/6	是否到剛剛標的第 1 格
4	COUNT 4/6	完整範圍已通過，才測第 4 格
3	COUNT 3/6	是否能回退一格

OLED 上的 ANGLE CMD 是命令，不是量角器。畫面顯示 4、指針仍停在 3，就記下這個差異，先停止再排查。

資訊可能已更新，但機構還在移動，也可能電源或接線有問題。不把「螢幕有字」當作整個作品完成。

三種停止，並不一樣

做法	程式狀態	外部電源	下一步
送 q	DISARMED · 停止脈波	可能仍 ON	外部 OFF · 再拔 USB
按 STOP 或送 x	ABORTED · 鎖定	可能仍 ON	外部 OFF · 查原因
外部 OFF · 再拔 USB	兩種供電關閉	OFF	才能改線或拆指針

中止後，不會自己繼續

解除原因、外部 OFF、STOP 放開，稍等後依序送：

c → 清除可清除的原因
 z → 回到 DISARMED · 還沒有重新動作
 a → 依已驗證上電流程重新準備第 0 格

按住 STOP 時，c / z / a 不能讓舵機重新啟用。a 每次重設 count 為 0，不是接續上次值。

停止控制訊號可能失去保持力，但不能保證立刻停止所有機械運動。軟體按鈕不是取代可操作的電源開關。

逾時與故障，不靠拔線測

一次只做一項。使用已確認的小範圍與輕指針；不故意卡住舵機，也不帶電拔 OLED。

測試	怎麼做	預期證據
太快輸入	兩個數字相隔不到 1 秒	第二個拒絕，count 不變
最長啟用時間	從 a 起算，不延長測試	到 30 秒中止， arm_session_timeout
軟體故障	送 f	injected_fault，停止脈波並 鎖定
STOP 持續按住	按住時嘗試 a	仍中止，不恢復

30 秒是本課程式的限制，不是舵機一定能安全工作的承諾；有異常應立刻關電源。

顯示通訊阻塞、函式執行及迴圈排程都會影響停止反應。本程式在顯示前後檢查，不能宣稱毫秒級最壞停止時間已經實測。

RST 不能當作恢復捷徑。重啟行為留到課前專門驗證；課堂先外部 OFF 再重啟，不在帶載時試未知瞬態。

這些紀錄能證明什麼？

以下都是教學情境，不是今天的實測結果。先寫答案，再翻頁。

1. 數字預覽用 60 ~ 120 對應 0 ~ 6。輸入 4，示例角度是多少？需要舵機嗎？
2. 外部電源 OFF，但 Serial 印出 `signal_attached=true`，可以說馬達已轉動嗎？
3. 按 STOP 後顯示 ABORTED，能立即拔線嗎？還缺哪兩個動作？
4. 假設 a 在 1000 ms 接受，4 在 2000 ms 接受。30 秒期限應到 31000，還是 32000 ms？
5. OLED 顯示 COUNT 4/6，紙指針卻在第 3 格。你會記錄什麼？第一個操作是什麼？

討論時不要只說「有問題」

指出你看到的欄位、它能支持的結論，以及還沒量到什麼。例如「命令被接受，但沒有真實位置資料」。

先把命令和量測分開

題	參考解答
1	$60 + 4 \times 10 = 100$ 。只用裸板與 Serial，不需要舵機。
2	不行。attached 表示控制通道啟用；沒有量到電源或轉軸位置。
3	不行。先外部電源 OFF，再拔 USB；確認斷電後才改線。
4	31000 ms 。期限從 a 算起，後續數字命令不延長。
5	同時記下 count / 命令角度及實際指針位置與時間。先中止並外部 OFF，不先加大角度或電壓。

第 5 題不能僅憑這兩個數字就判定「舵機壞了」。可能尚在移動、紙刻度或舵盤偏移，也可能供電或訊號有問題；必須分開檢查。

接著把 Week 2 的加減計數規則，加到已完成的指針作品上。

不用每次自己算下一個數字

Week 2 用按鈕讓數字加一、減一。這次先用 **Serial 的 + 與 - 命令** 做同樣的規則，再用 OLED 和紙指針表示結果。



這個版本**沒有新增兩顆實體加減按鈕**。既有按鈕繼續作 STOP，避免把停止功能挪走。用鍵盤可先專心核對計數、邊界與動作命令。

換上附錄「加減計數指針」完整程式。換程式仍要外部 OFF、拔 USB、移除外接線，不能帶載上傳。

同樣依已確認設定填入、先不開 OLED 測核心，再加入顯示。供電、停止、間隔與 30 秒上限都保留，不因改成計數器而解除。

實機範圍未完成時，只可做主機假輸入測試；不能把預覽程式的數值直接貼成舵機設定。

+、- 與上下限

完成原有 a 啟用流程。每次有效命令相隔至少 1 秒，先預測 count，再觀察。

起始值	送出的命令	預期 count
0	+	1
1	+	2
2	-	1
0	-	仍 0，拒絕低於下限
6	+	仍 6，拒絕超過上限

表格最後兩列是不同的起始情境，不是從上一列直接接續。

每次核對：Serial count、OLED count、紙盤位置。記錄「命令角度」，不要填成「實測角度」。

中止後再送 + 不應加分也不應繼續動作。恢復並重新 a 會歸零；這份程式沒有永久保存計數。

同一秒貼上很多 + 不是速度測試：超過接受頻率的命令會被丟棄，不會排隊等它慢慢轉。

原本的安全限制繼續用

程式原本接受 0~6。新版本先把 + / - 換算為下一個數字，再交回原本的條件檢查。

```
if (c == '+' || c == '-') {  
    if (count == 6 && c == '+') {  
        Serial.println("event_type=command_rejected reason=upper_limit");  
        return;  
    }  
    if (count == 0 && c == '-') {  
        Serial.println("event_type=command_rejected reason=lower_limit");  
        return;  
    }  
    c = char('0' + count + (c == '+' ? 1 : -1));  
}
```

'0' + 數字 把 0~6 轉回字元，沿用原本 handleCommand 的後續判斷。這段本身沒有直接呼叫 `pointer.write()`。

只有狀態允許、STOP 放開、沒有故障、距前次至少一秒，才接受下一個 count。拒絕的操作不修改 count。

每次改兩格，但不超出 0~6

使用「加減計數指針」程式，只改 + / - 的計數換算。不要修改供電設定、實體角度端點、STOP、1 秒間隔或 30 秒上限。

新規則

- +：加 2，但最多 6。
- -：減 2，但最少 0。
- 到上限再加、到下限再減，仍維持原本的拒絕紀錄。
- 先在紙上算或主機假輸入測試；完整安全行程已驗證才做實物。

要留下的證據

1. 從 0 依序 + + + + - - - -，每步預期值。
2. 先送 3，再 + + -，每步預期值。
3. ABORTED 時按 +，計數與脈波應有何結果？
4. 程式只改哪一段？為什麼不需要擴大舵機角度？

題目使用同一個已驗證的七格區間，不增加採購、負載或行程。先自己完成，下一頁再看參考解答。

只改下一個數字的計算

在前面的 + / - 處理區塊，保留兩個上下限拒絕條件，把最後的 `c = char(...)` 一行換成：

```
int next = count + (c == '+' ? 2 : -2);
if (next > 6) next = 6;
if (next < 0) next = 0;
c = char('0' + next);
```

測試	預期結果 (每次間隔符合要求)
從 0 : + + + +	2、4、6、6；第 4 次拒絕
接著：- - - -	4、2、0、0；第 4 次拒絕
先指定 3 : + + -	5、6、4
ABORTED 時 +	count 不變，不重新啟用脈波

仍然只傳 0~6 給原有換算函式，因此不必改角度端點。完整程式仍由原有條件決定是否接受，不直接繞過 `applyCount()` 的呼叫條件。

這些是參考預期與主機測試目標，不是舵機實測紀錄。改動後重新編譯，依相同斷電與上傳流程處理。

今天留下什麼？

正常結束送 q，外部 OFF，拔 USB，電表 OFF。確認完全斷電才收線；不要用手硬扳指針回零。

留下的資料	請寫具體內容
器材及接線	板卡、舵機版本、GPIO、電源方案、功能端點
命令與結果	一筆被接受、一筆被拒絕、一筆停止紀錄
畫面與位置	OLED / Serial 值與真正紙盤位置分開記
限制	尚未完成的供電、實機或負載測試

下週才把「手動給的數字」換成遊戲規則產生的有效次數。不是把今天的紙盤當成已完成的自動遊戲，也不需要今天先接回所有感測器。

排錯順序

設定缺值 → 先補來源；STOP 無效 → 先查斷電接線；有命令沒動 → 先停再查電源與訊號；紙盤偏位 → 檢查固定與刻度，不先擴大角度。

能說清楚「程式要求什麼、實物發生什麼、哪些還沒量到」，比只有一段轉動影片更完整。

查得到的資料與尚未確認的部分

- [ESP32Servo 原始碼與 API 說明](#)：attach、write、detach、read 的功能。教材固定使用既有 3.2.1 編譯基準，不自動升級。
- [TowerPro SG90 產品頁](#)：原廠特定產品的資料，不能證明所有同名商品都是同款。
- [TI LM2596 資料](#)：降壓晶片原理與限制；晶片能力不等於市售整片模組與線材的能力。
- [U8g2 專案](#)：OLED 函式及控制器選項；本週沿用 Week 5 的模組確認結果。

實物照片為課程既有照片；接線與訊號圖為教學示意，不按外形比例，也不代表插針的左右順序。硬體限制依課程截至 2026-09-07 的實測交接及後續軟體紀錄整理。

本重設稿完成的是文件、程式與非實物檢查。沒有新增舵機上電、負載、角度、共地或停止的實體測試。

下一段是完整程式附錄：數字預覽、受限紙指針、加減計數指針。每支程式分頁只是排版，合起來才是完整內容。

count_preview

第 1 ~ 29 行。所有分頁合起來才是完整程式。

```
// Numerical preview only: no GPIO, servo library, or actuator output.
const int ZERO_EXAMPLE = 60;
const int SIX_EXAMPLE = 120;
int count = 0;

int exampleAngle(int n) {
    return ZERO_EXAMPLE + (SIX_EXAMPLE - ZERO_EXAMPLE) * n / 6;
}

void previewCommand(char c) {
    if (c == '\r' || c == '\n' || c == ' ') return;
    if (c < '0' || c > '6') {
        Serial.println("event=rejected reason=enter_one_digit_0_to_6");
        return;
    }
    count = c - '0';
    Serial.printf("mode=preview count=%d angle_example=%d servo_connected=false\n",
        count, exampleAngle(count));
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("mode=preview enter_one_digit_0_to_6 no_servo_output");
}

void loop() {
    if (Serial.available()) previewCommand((char)Serial.read());
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。

week06_servo_pointer

第 1 ~ 30 行。所有分頁合起來才是完整程式。

```
#include <ESP32Servo.h>
#include <cstring>
#include <Wire.h>
#include <U8g2lib.h>

const char* DEVICE_ID = "BOARD-T01";
const bool SERVO_PROFILE_CONFIRMED = false;
const bool POWER_AND_FIXTURE_CONFIRMED = false;
const bool STOP_PROFILE_CONFIRMED = false;
const int PIN_SERVO = -1;
const int PIN_STOP = -1;
const int SERVO_HZ = -1;
const int SERVO_MIN_US = -1;
const int SERVO_MAX_US = -1;
const int SAFE_MIN_ANGLE = -1;
const int SAFE_MAX_ANGLE = -1;
const int ZERO_ANGLE = -1;
const int SIX_ANGLE = -1;
const bool WITH_OLED = false; // Enable only after the standalone pointer test.
const bool OLED_PROFILE_CONFIRMED = false;
const int PIN_SDA = -1;
const int PIN_SCL = -1;
const int OLED_ADDRESS_7BIT = -1;
// ACK is not a controller ID. Select after electrical and fixed-screen checks.
#ifndef OLED_CONTROLLER
#define OLED_CONTROLLER 1306
#endif
#if OLED_CONTROLLER == 1306
U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE, PIN_SCL, PIN_SDA);
#elif OLED_CONTROLLER == 1315
```

下一頁接續同一支程式，不另開草稿。

week06_servo_pointer

第 31 ~ 56 行。所有分頁合起來才是完整程式。

```
U8G2_SSD1315_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE, PIN_SCL, PIN_SDA);
#else
#error Unsupported OLED_CONTROLLER: use a verified 1306 or 1315 configuration.
#endif
Servo pointer;
const uint32_t MAX_ARM_MS = 30000;
const uint32_t COMMAND_SPACING_MS = 1000;
const uint32_t RELEASE_STABLE_MS = 30;
const uint32_t DISPLAY_INTERVAL_MS = 250;
enum PointerState { DISARMED, ARMED, ABORTED };
PointerState state = DISARMED;
bool ready=false, injectedFault=false, ioFault=false, cleared=false, changed=true;
bool stopPressed=true;
uint32_t releasedSince=0, armedAt=0, commandedAt=0, lastDraw=0;
int count=0, angleCommand=-1;
const char* reason="boot_no_signal";

bool profileReady() {
    if (!SERVO_PROFILE_CONFIRMED || !POWER_AND_FIXTURE_CONFIRMED ||
    !STOP_PROFILE_CONFIRMED) return false;
    if (PIN_SERVO < 0 || PIN_STOP < 0 || PIN_SERVO == PIN_STOP ||
        SERVO_HZ < 10 || SERVO_HZ > 400 || SERVO_MIN_US < 500 || SERVO_MAX_US > 2500 ||
        SERVO_MIN_US >= SERVO_MAX_US || SAFE_MIN_ANGLE < 0 || SAFE_MAX_ANGLE > 180 ||
        SAFE_MIN_ANGLE > ZERO_ANGLE || ZERO_ANGLE >= SIX_ANGLE || SIX_ANGLE >
    SAFE_MAX_ANGLE) return false;
    if (WITH_OLED) {
        if (!OLED_PROFILE_CONFIRMED || OLED_ADDRESS_7BIT < 8 || OLED_ADDRESS_7BIT > 119)
        return false;
        const int pins[]={PIN_SERVO,PIN_STOP,PIN_SDA,PIN_SCL};
```

下一頁接續同一支程式，不另開草稿。

week06_servo_pointer

第 57 ~ 81 行。所有分頁合起來才是完整程式。

```

    for(int i=0;i<4;i++) { if(pins[i]<0)return false;for(int
j=0;j<i;j++)if(pins[i]==pins[j])return false; }
    }
    return true;
}
int angleForCount(int n) { return ZERO_ANGLE + (SIX_ANGLE-ZERO_ANGLE)*n/6; }
const char* stateName() { return
state==ARMED?"ARMED":state==ABORTED?"ABORTED":"DISARMED"; }
bool released(uint32_t now) { return !stopPressed && now-
releasedSince>=RELEASE_STABLE_MS; }
bool oledAck() { Wire.beginTransmission(OLED_ADDRESS_7BIT);return
Wire.endTransmission()==0; }
void stopSignal(const char* why, bool latch) {
    if(pointer.attached())pointer.detach();
    // Do not leave a driven HIGH on an unpowered servo's signal input.
    digitalWrite(PIN_SERVO, LOW);pinMode(PIN_SERVO,OUTPUT);
    state=latch?ABORTED:DISARMED;reason=why;cleared=false;changed=true;
}
void serviceStop(uint32_t now) {
    stopPressed=digitalRead(PIN_STOP)==LOW;
    if(stopPressed) {
        releasedSince=now;
        if(state!=ABORTED ||
std::strcmp(reason,"physical_stop")!=0)stopSignal("physical_stop",true);
    }
    if(state==ARMED && now-armedAt>=MAX_ARM_MS)stopSignal("arm_session_timeout",true);
}
void applyCount(int next,uint32_t now) {

count=next;angleCommand=angleForCount(count);pointer.write(angleCommand);commandedAt=no
w;changed=true;
    Serial.printf("device_id=%s event_type=pointer_command value=%d unit=count
angle_command=%d "

```

下一頁接續同一支程式，不另開草稿。

week06_servo_pointer

第 82 ~ 107 行。所有分頁合起來才是完整程式。

```

        "position_measured=false uptime_ms=%lu\n",DEVICE_ID,count,angleCommand,
(unsigned long)now);
    }
    void handleCommand(char c,uint32_t now) {
        if(c=='\r' || c=='\n' || c==' ')return;
        if(c=='x'){stopSignal("manual_abort",true);return;}
        if(c=='f'){injectedFault=true;stopSignal("injected_fault",true);return;}
        if(c=='q' && state!=ABORTED)
    {stopSignal("manual_detach_power_not_removed",false);return;}
        if(c=='c' && state==ABORTED && released(now)) {
            injectedFault=false;ioFault=WITH_OLED&&!oledAck();cleared=!ioFault;

reason=cleared?"cause_cleared_reset_required":"display_still_unresponsive";changed=true
;return;
        }
        if(c=='z' && state==ABORTED && cleared && released(now) && !injectedFault &&
!ioFault){
            state=DISARMED;reason="reset_no_signal";cleared=false;changed=true;return;
        }
        // 'a' is the operator's acknowledgement that external power is OFF.
        // There is no voltage sensor here; software cannot verify this condition.
        if(c=='a' && state==DISARMED && released(now) && !injectedFault && !ioFault){

pointer.setPeriodHertz(SERVO_HZ);pointer.attach(PIN_SERVO,SERVO_MIN_US,SERVO_MAX_US);
        if(!pointer.attached()){stopSignal("attach_failed",true);return;}
        state=ARMED;armedAt=now;reason="operator_arm_power_off_acknowledged";
        applyCount(0,now);return;
        }
        if(c>='0' && c<='6' && state==ARMED && released(now) && !injectedFault && !ioFault &&
now-commandedAt>=COMMAND_SPACING_MS){applyCount(c-'0',now);return;}
        Serial.printf("event_type=command_rejected command=%c state=%s
reason=state_or_guard\n",c,stateName());
    }

```

下一頁接續同一支程式，不另開草稿。

week06_servo_pointer

第 108 ~ 133 行。所有分頁合起來才是完整程式。

```
void drawSnapshot(uint32_t now) {
    (void)now;
    if(!WITH_OLED||ioFault)return;
    if(!oledAck()){ioFault=true;stopSignal("display_ack_failed",true);return;}
    char text[24];oled.clearBuffer();oled.setFont(u8g2_font_6x12_tf);
    oled.drawStr(0,12,stateName());
    snprintf(text,sizeof(text),"COUNT %d/6",count);oled.drawStr(0,28,text);
    snprintf(text,sizeof(text),"ANGLE CMD %d",angleCommand);oled.drawStr(0,44,text);
    oled.drawStr(0,60,pointer.attached()?"SIGNAL ON / NO SENSE":"NO SIGNAL / NO SENSE");
    oled.sendBuffer();
    if(!oledAck()){ioFault=true;stopSignal("display_ack_failed",true);}
}

void setup(){
    Serial.begin(115200);delay(1000);
    if(!profileReady()){Serial.println("week=6 status=blocked
reason=profile_missing");return;}

    ready=true;digitalWrite(PIN_SERVO,LOW);pinMode(PIN_SERVO,OUTPUT);pinMode(PIN_STOP,INPUT
_PULLUP);
    releasedSince=millis();serviceStop(millis());
    if(WITH_OLED){
        if(!Wire.begin(PIN_SDA,PIN_SCL,100000))
        {ioFault=true;stopSignal("i2c_begin_failed",true);}
        else {Wire.setTimeout(20);if(!oledAck())
        {ioFault=true;stopSignal("display_ack_failed",true);}
        else
        {oled.setI2CAddress(OLED_ADDRESS_7BIT*2);oled.setBusClock(100000);oled.begin();Wire.set
        Timeout(20);}}
    }
    Serial.println("week=6 commands=a,0..6,q,x,f,c,z
external_power=operator_checked_not_sensed");
}

void loop(){
    if(!ready)return;
```

下一頁接續同一支程式，不另開草稿。

week06_servo_pointer

第 134 ~ 143 行。所有分頁合起來才是完整程式。

```
uint32_t now=millis();serviceStop(now);
if(Serial.available())handleCommand((char)Serial.read(),now);
if(now-lastDraw>=DISPLAY_INTERVAL_MS){lastDraw=now;drawSnapshot(now);}
now=millis();serviceStop(now);
if(changed){changed=false;
    Serial.printf("device_id=%s event_type=pointer_status state=%s value=%d unit=count
"
                "signal_attached=%s position_measured=false power_measured=false
reason=%s uptime_ms=%lu\n",
                DEVICE_ID,stateName(),count,pointer.attached()?"true":"false",reason,
(unsigned long)now);
    }
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。

serial_pointer_counter

第 1 ~ 31 行。所有分頁合起來才是完整程式。

```
// Extension of the course pointer: Serial + / - changes count.
// Physical STOP, timeout and all original power/profile guards are retained.
#include <ESP32Servo.h>
#include <cstring>
#include <Wire.h>
#include <U8g2lib.h>

const char* DEVICE_ID = "BOARD-T01";
const bool SERVO_PROFILE_CONFIRMED = false;
const bool POWER_AND_FIXTURE_CONFIRMED = false;
const bool STOP_PROFILE_CONFIRMED = false;
const int PIN_SERVO = -1;
const int PIN_STOP = -1;
const int SERVO_HZ = -1;
const int SERVO_MIN_US = -1;
const int SERVO_MAX_US = -1;
const int SAFE_MIN_ANGLE = -1;
const int SAFE_MAX_ANGLE = -1;
const int ZERO_ANGLE = -1;
const int SIX_ANGLE = -1;
const bool WITH_OLED = false; // Enable only after the standalone pointer test.
const bool OLED_PROFILE_CONFIRMED = false;
const int PIN_SDA = -1;
const int PIN_SCL = -1;
const int OLED_ADDRESS_7BIT = -1;
// ACK is not a controller ID. Select after electrical and fixed-screen checks.
#ifndef OLED_CONTROLLER
#define OLED_CONTROLLER 1306
#endif
#if OLED_CONTROLLER == 1306
U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE, PIN_SCL, PIN_SDA);
```

下一頁接續同一支程式，不另開草稿。

serial_pointer_counter

第 32 ~ 58 行。所有分頁合起來才是完整程式。

```
#elif OLED_CONTROLLER == 1315
U8G2_SSD1315_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE, PIN_SCL, PIN_SDA);
#else
#error Unsupported OLED_CONTROLLER: use a verified 1306 or 1315 configuration.
#endif

Servo pointer;
const uint32_t MAX_ARM_MS = 30000;
const uint32_t COMMAND_SPACING_MS = 1000;
const uint32_t RELEASE_STABLE_MS = 30;
const uint32_t DISPLAY_INTERVAL_MS = 250;
enum PointerState { DISARMED, ARMED, ABORTED };
PointerState state = DISARMED;
bool ready=false, injectedFault=false, ioFault=false, cleared=false, changed=true;
bool stopPressed=true;
uint32_t releasedSince=0, armedAt=0, commandedAt=0, lastDraw=0;
int count=0, angleCommand=-1;
const char* reason="boot_no_signal";

bool profileReady() {
    if (!SERVO_PROFILE_CONFIRMED || !POWER_AND_FIXTURE_CONFIRMED ||
    !STOP_PROFILE_CONFIRMED) return false;
    if (PIN_SERVO < 0 || PIN_STOP < 0 || PIN_SERVO == PIN_STOP ||
        SERVO_HZ < 10 || SERVO_HZ > 400 || SERVO_MIN_US < 500 || SERVO_MAX_US > 2500 ||
        SERVO_MIN_US >= SERVO_MAX_US || SAFE_MIN_ANGLE < 0 || SAFE_MAX_ANGLE > 180 ||
        SAFE_MIN_ANGLE > ZERO_ANGLE || ZERO_ANGLE >= SIX_ANGLE || SIX_ANGLE >
        SAFE_MAX_ANGLE) return false;
    if (WITH_OLED) {
        if (!OLED_PROFILE_CONFIRMED || OLED_ADDRESS_7BIT < 8 || OLED_ADDRESS_7BIT > 119)
        return false;
        const int pins[]={PIN_SERVO,PIN_STOP,PIN_SDA,PIN_SCL};
```

下一頁接續同一支程式，不另開草稿。

serial_pointer_counter

第 59 ~ 83 行。所有分頁合起來才是完整程式。

```
    for(int i=0;i<4;i++) { if(pins[i]<0)return false;for(int
j=0;j<i;j++)if(pins[i]==pins[j])return false; }
    }
    return true;
}
int angleForCount(int n) { return ZERO_ANGLE + (SIX_ANGLE-ZERO_ANGLE)*n/6; }
const char* stateName() { return
state==ARMED?"ARMED":state==ABORTED?"ABORTED":"DISARMED"; }
bool released(uint32_t now) { return !stopPressed && now-
releasedSince>=RELEASE_STABLE_MS; }
bool oledAck() { Wire.beginTransmission(OLED_ADDRESS_7BIT);return
Wire.endTransmission()==0; }
void stopSignal(const char* why, bool latch) {
    if(pointer.attached())pointer.detach();
    // Do not leave a driven HIGH on an unpowered servo's signal input.
    digitalWrite(PIN_SERVO, LOW);pinMode(PIN_SERVO,OUTPUT);
    state=latch?ABORTED:DISARMED;reason=why;cleared=false;changed=true;
}
void serviceStop(uint32_t now) {
    stopPressed=digitalRead(PIN_STOP)==LOW;
    if(stopPressed) {
        releasedSince=now;
        if(state!=ABORTED ||
std::strcmp(reason,"physical_stop")!=0)stopSignal("physical_stop",true);
    }
    if(state==ARMED && now-armedAt>=MAX_ARM_MS)stopSignal("arm_session_timeout",true);
}
void applyCount(int next,uint32_t now) {

count=next;angleCommand=angleForCount(count);pointer.write(angleCommand);commandedAt=no
w;changed=true;
    Serial.printf("device_id=%s event_type=pointer_command value=%d unit=count
angle_command=%d "

```

下一頁接續同一支程式，不另開草稿。

serial_pointer_counter

第 84 ~ 111 行。所有分頁合起來才是完整程式。

```
        "position_measured=false uptime_ms=%lu\n",DEVICE_ID,count,angleCommand,
(unsigned long)now);
}
void handleCommand(char c,uint32_t now) {
    if(c=='\r' || c=='\n' || c==' ')return;
    if(c=='x'){stopSignal("manual_abort",true);return;}
    if(c=='f'){injectedFault=true;stopSignal("injected_fault",true);return;}
    if(c=='q' && state!=ABORTED)
{stopSignal("manual_detach_power_not_removed",false);return;}
    if(c=='c' && state==ABORTED && released(now)) {
        injectedFault=false;ioFault=WITH_OLED&&!oledAck();cleared=!ioFault;

reason=cleared?"cause_cleared_reset_required":"display_still_unresponsive";changed=true
;return;
    }
    if(c=='z' && state==ABORTED && cleared && released(now) && !injectedFault &&
!ioFault){
        state=DISARMED;reason="reset_no_signal";cleared=false;changed=true;return;
    }
    // 'a' is the operator's acknowledgement that external power is OFF.
    // There is no voltage sensor here; software cannot verify this condition.
    if(c=='a' && state==DISARMED && released(now) && !injectedFault && !ioFault){

pointer.setPeriodHertz(SERVO_HZ);pointer.attach(PIN_SERVO,SERVO_MIN_US,SERVO_MAX_US);
        if(!pointer.attached()){stopSignal("attach_failed",true);return;}
        state=ARMED;armedAt=now;reason="operator_arm_power_off_acknowledged";
        applyCount(0,now);return;
    }
    if(c=='+' || c=='-') {
        if(count==6 && c=='+') {
            Serial.println("event_type=command_rejected reason=upper_limit");return;
        }
        if(count==0 && c=='-') {
            Serial.println("event_type=command_rejected reason=lower_limit");return;
        }
    }
}
```

下一頁接續同一支程式，不另開草稿。

serial_pointer_counter

第 112 ~ 137 行。所有分頁合起來才是完整程式。

```

    }
    c=char('0'+count+(c=='+'?1:-1));
  }
  if(c>='0' && c<='6' && state==ARMED && released(now) && !injectedFault && !ioFault &&
    now-commandedAt>=COMMAND_SPACING_MS){applyCount(c-'0',now);return;}
  Serial.printf("event_type=command_rejected command=%c state=%s
reason=state_or_guard\n",c,stateName());
}
void drawSnapshot(uint32_t now) {
  (void)now;
  if(!WITH_OLED||ioFault)return;
  if(!oledAck()){ioFault=true;stopSignal("display_ack_failed",true);return;}
  char text[24];oled.clearBuffer();oled.setFont(u8g2_font_6x12_tf);
  oled.drawStr(0,12,stateName());
  snprintf(text,sizeof(text),"COUNT %d/6",count);oled.drawStr(0,28,text);
  snprintf(text,sizeof(text),"ANGLE CMD %d",angleCommand);oled.drawStr(0,44,text);
  oled.drawStr(0,60,pointer.attached()?"SIGNAL ON / NO SENSE":"NO SIGNAL / NO SENSE");
  oled.sendBuffer();
  if(!oledAck()){ioFault=true;stopSignal("display_ack_failed",true);}
}
void setup(){
  Serial.begin(115200);delay(1000);
  if(!profileReady()){Serial.println("week=6 status=blocked
reason=profile_missing");return;}

  ready=true;digitalWrite(PIN_SERVO,LOW);pinMode(PIN_SERVO,OUTPUT);pinMode(PIN_STOP,INPUT
_PULLUP);
  releasedSince=millis();serviceStop(millis());
  if(WITH_OLED){
    if(!Wire.begin(PIN_SDA,PIN_SCL,100000))
    {ioFault=true;stopSignal("i2c_begin_failed",true);}
  }

```

下一頁接續同一支程式，不另開草稿。

serial_pointer_counter

第 138 ~ 154 行。所有分頁合起來才是完整程式。

```
        else {Wire.setTimeout(20);if(!oledAck())
{ioFault=true;stopSignal("display_ack_failed",true);}
        else
{oled.setI2CAddress(OLED_ADDRESS_7BIT*2);oled.setBusClock(100000);oled.begin();Wire.set
Timeout(20);}}
    }
    Serial.println("week=6 commands=a,0..6,+,-,q,x,f,c,z
external_power=operator_checked_not_sensed");
}
void loop(){
    if(!ready)return;
    uint32_t now=millis();serviceStop(now);
    if(Serial.available())handleCommand((char)Serial.read(),now);
    if(now-lastDraw>=DISPLAY_INTERVAL_MS){lastDraw=now;drawSnapshot(now);}
    now=millis();serviceStop(now);
    if(changed){changed=false;
        Serial.printf("device_id=%s event_type=pointer_status state=%s value=%d unit=count
"
                        "signal_attached=%s position_measured=false power_measured=false
reason=%s uptime_ms=%lu\n",
                        DEVICE_ID,stateName(),count,pointer.attached()?"true":"false",reason,
(unsigned long)now);
    }
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。